CEBU INSTITUTE OF TECHNOLOGY UNIVERSITY

COLLEGE OF COMPUTER STUDIES

Software Test Document

for

ScheduleSkies

Signature

Labiste, Vince Jerby P.

Member 1

Medalla, Mon Vencie

Member 2

Pescadero, Christian L.

Member 3

Susan, Joshlee Rash

Member 4

Villamor, John Michael

Member 5

Change History

None.

Preface

This is the System Test Description (STD) document for the ScheduleSkies project. In particular, the document details the test description of the software. It contains the introduction, test plan, and test case specification.

Table of Contents

Signature 2
Change History 3
Preface 4
Table of Contents 5
List of Figures 7
List of Tables 8
Introduction 9
1.1 System Overview 9
1.2 Test Approach 9
1.3 Definitions and Acronyms 9
Test Plan 11
2.1 Features to be Tested 11
2.2 Features not to be Tested 11
2.3 Testing Tools and Environment 11
Test Cases 12
3.1 Case-1 – User Registration – Valid Credentials 12
3.2 Case-2 – User Registration – Invalid Email Format 12
3.3 Case-3 – User Login – Valid Credentials 12
3.4 Case-4 – User Login – Invalid Password 13
3.5 Case-5 – User Logout 13
3.6 Case-6 – Create New Itinerary 14
3.7 Case-7 – Edit Existing Itinerary 14
3.8 Case-8 – Delete Itinerary 15
3.9 Case-9 – Add Destination to Itinerary 15
3.10 Case-10 – Overlapping Schedule Conflict Detection 16
3.11 Case-11 – Weather API Data Retrieval 16
3.12 Case-12 – Weather-Based Activity Warning 17
3.13 Case-13 – Google Maps Location Search 17
3.14 Case-14 – Route Estimation Display 17
3.15 Case-15 – AI Structured Recommendation Generation 18
3.16 Case-16 – AI Weather-Aware Adjustment Logic 18
3.17 Case-17 – Invite Collaborator to Itinerary 19
3.18 Case-18 – Multi-User Editing Synchronization 19
3.19 Case-19 – Voting Feature Functionality 19
3.20 Case-20 – Duplicate Voting Prevention 20
3.21 Case-21 – Closed Location Warning Detection 20
3.22 Case-22 – Session Timeout Validation 21
3.23 Case-23 – Account Deletion Functionality 21
3.24 Case-24 – Privacy Consent Enforcement 22
3.25 Case-25 – Unauthorized Dashboard Access Prevention 22
3.26 Case-26 – Required Field Validation (Itinerary Creation) 22
3.27 Case-27 – Weather API Failure Handling 23
3.28 Case-28 – Google Chrome Compatibility Testing 23
3.29 Case-29 – Microsoft Edge Compatibility Testing 24
3.30 Case-30 – Mobile Responsive Layout Validation 24
3.31 Case-31 – Map Zoom and Interaction Validation 24
3.32 Case-32 – Travel Analytics Storage Verification 25
3.33 Case-33 – Real-Time Location Non-Persistence Validation 25
3.34 Case-34 – CRUD Operation Unit Testing 26
3.35 Case-35 – Weather Data Parsing Unit Test 26
3.36 Case-36 – AI Response Formatter Unit Test 26
3.37 Case-37 – Dashboard Rendering Validation 27
3.38 Case-38 – Navigation Routing Verification 27
3.39 Case-39 – Database Integrity Validation 27
3.40 Case-40 – HTTPS Secure Connection Enforcement 28
Appendix (Test Logs) 29
A.1 Log for Test n 29
A.2 Test Results 29
A.3 Incident Report 30

List of Figures

List of Tables

Introduction

1.1 System Overview

ScheduleSkies is a smart web-based itinerary planning application that integrates weather forecasting, map routing, AI-generated recommendations, and collaborative scheduling. The system allows authenticated users to create, edit, and manage travel plans while dynamically adjusting activities based on real-time environmental conditions. The application is deployed in a localhost environment using Supabase (Free Tier), Google Maps API, a weather API (Free Tier), and Gemini AI API (Free Tier). Testing focuses on validating functional requirements defined in the SRS and implemented in the SDD.

1.2 Test Approach

The testing strategy includes:

- Manual Functional Testing
- Basic Unit Testing (CRUD operations, API parsing, AI formatter)
- Localhost Deployment Testing
- Web and Mobile Browser Responsive Testing

All team members execute test cases. Defects are recorded in the Issue Logs as defined in the SPMP.

Testing focuses strictly on functional validation. Performance stress testing and security penetration testing are outside scope.

1.3 Definitions and Acronyms

AI – Artificial Intelligence

API – Application Programming Interface

CRUD – Create, Read, Update, Delete

ETA – Estimated Time of Arrival

RLS – Row-Level Security

STD – Software Test Document

SRS – Software Requirements Specification

SDD – Software Design Document

SPMP – Software Project Management Plan

Test Plan

This test plan outlines the approach and activities used to verify the functionality and reliability of the ScheduleSkies system. It defines the testing scope, methods, and responsibilities to ensure the system meets its intended requirements before deployment.

2.1 Features to be Tested

Features of the ScheduleSkies system listed below shall be tested. These include working properly according the goals set before the project is finally presented to the client or panel of judges.

- User Registration and Login
- Authentication and Session Management
- Itinerary CRUD Operations
- Weather API Integration
- Weather-based Warnings
- Google Maps Integration
- Route Estimation
- AI-Generated Recommendations
- AI Weather Adjustments
- Collaboration (Invite, Multi-Edit, Voting)
- Conflict Detection (Overlapping Schedule)
- Closed Location Warnings
- Responsive Web Layout
- Input Validation
- Privacy Consent Enforcement
- Database Integrity
- Unit-Level Function Testing

2.2 Features not to be Tested

Features of the ScheduleSkies system listed below shall not be tested.

- Commercial payment processing (not implemented)
- Large-scale performance stress testing
- Full penetration security testing
- Third-party API internal functionality

2.3 Testing Tools and Environment

Software testing will be done using the following resources:

- Environment:
  - Localhost deployment
  - Supabase Free Tier
  - Google Chrome and Other available Web Browsers
  - Mobile Browser Responsive Mode
- Tools:
  - Visual Studio Code

Test Cases

3.1 Case-1 – User Registration – Valid Credentials

Purpose

Verify successful user registration with valid credentials.

Inputs

Condition: Valid email and password

Input: user@example.com / StrongPassword123

Expected Outputs & Pass/Fail Criteria

Output: Account created and redirected to dashboard
Pass: User stored in database and redirected
Fail: Registration error occurs

Test Procedure

1. Open registration page
2. Enter valid credentials
3. Click Register
4. Confirm dashboard loads

3.2 Case-2 – User Registration – Invalid Email Format

Purpose

Validate email format enforcement.

Inputs

Input: invalid-email-format

Expected Outputs & Pass/Fail Criteria

Output: Error message displayed
Pass: Registration blocked
Fail: Account created

Test Procedure

1. Enter invalid email
2. Submit

3.3 Case-3 – User Login – Valid Credentials

Purpose

Verify successful login.

Inputs

Valid registered credentials

Expected Outputs & Pass/Fail Criteria

Output: Redirect to dashboard
Pass: Session initiated
Fail: Access denied

Test Procedure

1. Login using valid credentials

3.4 Case-4 – User Login – Invalid Password

Purpose

Verify login failure with incorrect password.

Inputs

Correct email + wrong password

Expected Outputs & Pass/Fail Criteria

Output: “Invalid credentials”
Pass: Access denied
Fail: Login successful

Procedure

1. Enter wrong password
2. Login

3.5 Case-5 – User Logout

Purpose

Verify logout functionality.

Inputs

Click Logout

Expected Outputs & Pass/Fail Criteria

Output: Redirect to login page
Pass: Session destroyed

Test Procedure

1. Click logout button

3.6 Case-6 – Create New Itinerary

Purpose

To verify that an authenticated user can successfully create a new itinerary.

Inputs

Condition: User is logged in

Input:

- Title: “Cebu Trip”
- Date: Valid future date
- Location: Cebu City

Expected Outputs & Pass/Fail Criteria

Output:

- Itinerary saved to database
- Itinerary appears on dashboard

Pass: Data stored correctly and displayed
Fail: Itinerary not saved or not visible

Test Procedure

1. Login
2. Click “Create Itinerary”
3. Enter valid details
4. Click Save
5. Verify itinerary appears.

3.7 Case-7 – Edit Existing Itinerary

Purpose

Verify that users can edit an existing itinerary.

Inputs

Condition: Existing itinerary present

Input: Modify title to “Cebu Summer Trip”

Expected Outputs & Pass/Fail Criteria

Output: Updated itinerary displayed
Pass: Changes reflected immediately
Fail: Old data persists / Error

Test Procedure

1. Select itinerary

2. Click Edit
3. Modify details
4. Save changes
5. Confirm update

3.8 Case-8 – Delete Itinerary

Purpose

Verify itinerary deletion.

Inputs

Click Delete on existing itinerary

Expected Outputs & Pass/Fail Criteria

Output: Itinerary removed from dashboard and database
Pass: No longer visible
Fail: Still appears

Test Procedure

1. Delete itinerary
2. Refresh page
3. Verify removal

3.9 Case-9 – Add Destination to Itinerary

Purpose

Verify adding a destination to itinerary.

Inputs

Input destination: “Temple of Leah”

Expected Outputs & Pass/Fail Criteria

Output: Destination added under itinerary
Pass: Visible in activity list
Fail: Not added

Procedure

1. Open itinerary
2. Add destination
3. Save

3.10 Case-10 – Overlapping Schedule Conflict Detection

Purpose

Verify detection of overlapping schedule entries.

Inputs

Add two activities with same time

Expected Outputs & Pass/Fail Criteria

Output: Conflict warning displayed

Pass: System prevents saving

Fail: Both entries saved without warning

Test Procedure

1. Create first activity
2. Create second same time
3. Save

3.11 Case-11 – Weather API Data Retrieval

Purpose

Verify weather API data retrieval.

Inputs

Location

Expected Outputs & Pass/Fail Criteria

Output: Weather forecast displayed
Pass: Data matches API response
Fail: No data or incorrect data

Test Procedure

1. Open itinerary
2. Select location
3. View weather

3.12 Case-12 – Weather-Based Activity Warning

Purpose

Verify weather-based warning system.

Inputs

Date with rain forecast

Expected Outputs & Pass/Fail Criteria

Output: Warning message displayed
Pass: Suggest reschedule or indoor plan
Fail: No warning shown

Test Procedure

1. Select rainy date
2. Observe alert

3.13 Case-13 – Google Maps Location Search

Purpose

Verify Google Maps location search.

Inputs

Search: “SM Seaside Cebu”

Expected Outputs & Pass/Fail Criteria

Output: Correct map pin displayed
Pass: Pin location accurate
Fail: Incorrect location

Test Procedure

1. Enter location in search
2. Confirm pin

3.14 Case-14 – Route Estimation Display

Purpose

Verify route estimation.

Inputs

Start: Current location
Destination: SM Seaside

Expected Outputs & Pass/Fail Criteria

Output: Route and ETA displayed

Pass: Route visible
Fail: No route

Procedure

1. Click route
2. Confirm display

3.15 Case-15 – AI Structured Recommendation Generation

Purpose

Validate AI structured recommendation.

Inputs

Generate AI suggestion

Expected Outputs & Pass/Fail Criteria

Output: Organized itinerary list

Pass: Structured format (bullets or sections)

Fail: Unstructured paragraph only

Test Procedure

1. Click Generate AI
2. Review output

3.16 Case-16 – AI Weather-Aware Adjustment Logic

Purpose

Validate AI weather adjustment logic.

Inputs

Rain forecast date

Expected Outputs & Pass/Fail Criteria

Output: Indoor alternatives suggested

Pass: Weather-aware recommendation

Fail: Outdoor suggestion remains

Test Procedure

1. Generate AI during rainy

3.17 Case-17 – Invite Collaborator to Itinerary

Purpose

Verify inviting collaborator.

Inputs

Enter collaborator email

Expected Outputs & Pass/Fail Criteria

Output: Access granted
Pass: Collaborator sees itinerary
Fail: No access

Test Procedure

1. Invite
2. Login as collaborator
3. Verify access

3.18 Case-18 – Multi-User Editing Synchronization

Purpose

Verify multi-user editing functionality.

Inputs

Two users edit itinerary

Expected Outputs & Pass/Fail Criteria

Output: Changes reflected
Pass: Latest version visible
Fail: Data overwritten incorrectly

Test Procedure

1. User A edits
2. User B refreshes
3. Confirm changes

3.19 Case-19 – Voting Feature Functionality

Purpose

Verify voting functionality.

Inputs

Vote option selected

Expected Outputs & Pass/Fail Criteria

Output: Vote count increments
Pass: Accurate tally
Fail: Incorrect count

Procedure

1. Cast vote
2. Verify count

3.20 Case-20 – Duplicate Voting Prevention

Purpose

To verify that the system prevents duplicate voting by the same user in a collaborative itinerary decision.

Inputs

Condition: User already voted once
Input: Same authenticated user attempts to vote again

Expected Outputs & Pass/Fail Criteria

Output: Second vote denied

Pass: Count unchanged

Fail: Count increases

Test Procedure

1. Login as User A
2. Navigate to collaborative itinerary
3. Cast vote on option
4. Attempt to vote again
5. Verify system blocks second vote
6. Confirm vote count remains the same

3.21 Case-21 – Closed Location Warning Detection

Purpose

To verify the system displays a warning when a selected destination is closed on the chosen date.

Inputs

Input: Select location known to be closed on selected date

Expected Outputs & Pass/Fail Criteria

Output: Closed-location alert message displayed
Pass: Warning shown before saving itinerary
Fail: No warning displayed

Test Procedure

1. Create or edit itinerary
2. Select location
3. Choose closed date
4. Attempt to save
5. Confirm warning appears

3.22 Case-22 – Session Timeout Validation

Purpose

To verify automatic session timeout after inactivity.

Inputs

Condition: User idle for extended period

Expected Outputs & Pass/Fail Criteria

Output: Redirect to login page
Pass: Session terminated
Fail: User remains logged in

Test Procedure

1. Login Leave system idle
2. Wait for timeout duration
3. Attempt navigation
4. Confirm redirection

3.23 Case-23 – Account Deletion Functionality

Purpose

Verify account deletion functionality.

Inputs

Click Delete Account and Confirm

Expected Outputs & Pass/Fail Criteria

Output: Account permanently removed
Pass: Cannot login again
Fail: Account still accessible

Test Procedure

1. Delete account
2. Attempt login
3. Verify denial

3.24 Case-24 – Privacy Consent Enforcement

Purpose

Verify privacy consent enforcement during registration.

Inputs

Uncheck consent checkbox

Expected Outputs & Pass/Fail Criteria

Output: Registration blocked
Pass: Cannot proceed without consent
Fail: Account created

Procedure

1. Attempt registration without checking consent

3.25 Case-25 – Unauthorized Dashboard Access Prevention

Purpose

Verify unauthorized dashboard access prevention.

Inputs

Directly enter dashboard URL without login

Expected Outputs & Pass/Fail Criteria

Output: Redirect to login
Pass: Access denied
Fail: Dashboard visible

Test Procedure

1. Logout
2. Enter dashboard URL
3. Confirm redirect

3.26 Case-26 – Required Field Validation (Itinerary Creation)

Purpose

Verify required field validation in itinerary creation.

Inputs

Leave Title field empty

Expected Outputs & Pass/Fail Criteria

Output: Error message displayed
Pass: Save blocked
Fail: Itinerary saved

Test Procedure

1. Attempt to save empty

3.27 Case-27 – Weather API Failure Handling

Purpose

Verify graceful handling of weather API failure.

Inputs

Simulated API disconnection

Expected Outputs & Pass/Fail Criteria

Output: Error message shown without system crash
Pass: Application remains stable
Fail: Application crashes

Test Procedure

1. Disconnect API
2. Refresh weather module

3.28 Case-28 – Google Chrome Compatibility Testing

Purpose

Verify compatibility in Google Chrome browser.

Inputs

Open application in Chrome

Expected Outputs & Pass/Fail Criteria

Output: UI loads correctly
Pass: No layout distortion
Fail: Broken UI elements

Test Procedure

1. Launch Chrome
2. Access localhost
3. Review pages

3.29 Case-29 – Microsoft Edge Compatibility Testing

Purpose

Verify compatibility in Microsoft Edge.

Inputs

Open application in Edge

Expected Outputs & Pass/Fail Criteria

Output: UI loads correctly
Pass: No layout distortion
Fail: Broken UI elements

Procedure

1. Launch Microsoft Edge
2. Access localhost
3. Review pages

3.30 Case-30 – Mobile Responsive Layout Validation

Purpose

Verify mobile responsive layout.

Inputs

Open responsive mobile view

Expected Outputs & Pass/Fail Criteria

Output: Layout adjusts dynamically
Pass: No overflow or distortion
Fail: UI elements overlap

Test Procedure

1. Open Developer Tools
2. Enable mobile mode

3.31 Case-31 – Map Zoom and Interaction Validation

Purpose

Verify map zoom and interaction.

Inputs

Use zoom controls

Expected Outputs & Pass/Fail Criteria

Output: Smooth zoom functionality
Pass: Map interactive
Fail: Frozen map

Test Procedure

1. Zoom in/out
2. Drag map

3.32 Case-32 – Travel Analytics Storage Verification

Purpose

Verify analytics storage of visited locations.

Inputs

Complete itinerary

Expected Outputs & Pass/Fail Criteria

Output: Data recorded in analytics
Pass: Stored correctly
Fail: No record stored

Test Procedure

1. Complete plan
2. Check database

3.33 Case-33 – Real-Time Location Non-Persistence Validation

Purpose

Verify that real-time location data is not stored long-term.

Inputs

Use live location feature

Expected Outputs & Pass/Fail Criteria

Output: No persistent location record saved
Pass: Database contains no location history
Fail: Location stored

Test Procedure

1. Use live tracking
2. Inspect database

3.34 Case-34 – CRUD Operation Unit Testing

Purpose

Unit Test: Validate CRUD functions.

Inputs

Call create, read, update, delete functions

Expected Outputs & Pass/Fail Criteria

Output: Correct function return values
Pass: All operations succeed
Fail: Error returned

Procedure

1. Run unit test script

3.35 Case-35 – Weather Data Parsing Unit Test

Purpose

Unit Test: Validate weather JSON parsing.

Inputs

Sample weather API response

Expected Outputs & Pass/Fail Criteria

Output: Correct temperature and condition extracted
Pass: Accurate parsing
Fail: Incorrect data mapping

Test Procedure

1. Run parsing function with mock data

3.36 Case-36 – AI Response Formatter Unit Test

Purpose

Unit Test: Validate AI response formatting.

Inputs

Raw Gemini API output

Expected Outputs & Pass/Fail Criteria

Output: Structured list format

Pass: Proper formatting applied

Fail: Unstructured output

Test Procedure

1. Run AI formatter function

3.37 Case-37 – Dashboard Rendering Validation

Purpose

Verify dashboard rendering of all widgets.

Inputs

Login to dashboard

Expected Outputs & Pass/Fail Criteria

Output: Weather, Map, Itinerary panels visible
Pass: All components load
Fail: Missing component

Test Procedure

1. Login
2. Inspect dashboard

3.38 Case-38 – Navigation Routing Verification

Purpose

Verify navigation routing.

Inputs

Click navigation menu links

Expected Outputs & Pass/Fail Criteria

Output: Correct pages load
Pass: No broken links
Fail: Incorrect routing

Test Procedure

1. Click each menu option

3.39 Case-39 – Database Integrity Validation

Purpose

Verify database integrity.

Inputs

Create and update itinerary

Expected Outputs & Pass/Fail Criteria

Output: Database reflects correct data
Pass: Matches user input
Fail: Data mismatch

Procedure

1. Perform CRUD
2. Inspect database

3.40 Case-40 – HTTPS Secure Connection Enforcement

Purpose

Verify HTTPS secure connection enforcement.

Inputs

Access system

Expected Outputs & Pass/Fail Criteria

Output: Secure HTTPS connection active
Pass: Browser shows secure connection
Fail: HTTP unsecured connection

Test Procedure

1. Access application
2. Inspect URL security indicator

Appendix (Test Logs)

A.1 Log for Test n

Test Case Number:

Date Executed:

Tester Name:

Status (Pass/Fail):

Remarks:

A.2 Test Results

Total Test Cases: 40

Passed: ___

Failed: ___

Pending: ___

A.3 Incident Report

Incident ID:

Test Case Reference:

Description of Issue:

Severity Level (Low/Medium/High/Critical):

Assigned To:

Resolution Status: